LEARN | PRACTICE | BUILD | MONITOR | GROW

VERIQTA

Prometheus HANDBOOK

A COMPLETE GUIDE FOR JUNIOR ENGINEERS

- ✓ Understand
- ✓ Install
- ✓ Configure
- ✓ Query with PromQL
- ✓ Monitor Real Systems
- ✓ Create Alerts
- ✓ Integrate with Grafana
- ✓ Use with Kubernetes
- ✓ Troubleshoot
- ✓ Build Production Skills

METRICS TODAY • RELIABLE SYSTEMS TOMORROW

INFRASTRUCTURE | APPLICATIONS | KUBERNETES | OBSERVABILITY | SRE | REAL WORLD SKILLS

VERIQTA

A VERIQTA LEARNING RESOURCE

Build Monitor Improve

VERIQTA

Monitoring, Metrics, and Why Prometheus Exists

A STRONG FOUNDATION FOR EVERY DEVOPS AND SRE ENGINEER

OBSERVE
UNDERSTAND
TROUBLESHOOT
KEEP SYSTEMS RUNNING

1. WHAT IS MONITORING?

Monitoring is the continuous collection and analysis of data from systems, applications and infrastructure to understand their health and performance.

It helps you detect problems early, understand what is happening, and take action before users are affected.

2. METRICS vs LOGS vs TRACES

METRICS

- Numerical data
- Collected over time
- Used for monitoring
- Example: CPU usage, memory, request rate
- **Answer:** How much? How many?

LOGS

- Text records
- Events and details
- Used for debugging
- Example: error messages, access logs
- **Answer:** What happened?

TRACES

- Track a request flow
- Show the path across services
- Used for performance analysis
- Example: request latency across microservices
- **Answer:** Where did it happen and how long did it take?

3. REACTIVE vs PROACTIVE MONITORING

REACTIVE

- You find out after something breaks
- Users are already affected
- You respond to incidents
- Higher risk and downtime

PROACTIVE

- You detect issues early
- You prevent problems
- You take action before users are affected
- Leads to more reliable systems

4. WHAT IS PROMETHEUS?

Prometheus is an open source monitoring and alerting toolkit designed for reliability and scale.

It collects time-series metrics from targets, stores them, allows powerful querying with PromQL, and integrates with Alertmanager for alerting.

5. WHAT PROBLEM PROMETHEUS SOLVES

Modern systems are dynamic and distributed. Prometheus solves the need for a reliable, scalable, and flexible monitoring solution that can:

- Collect metrics from many sources
- Store them efficiently as time series
- Allow fast querying and analysis
- Trigger alerts when things go wrong
- Integrate with modern environments like Kubernetes

6. TIME-SERIES MONITORING

Prometheus stores data as time series. A time series is a metric value recorded over time, identified by a metric name and a set of labels.

```
http_requests_total{method="GET",
instance="web1"}  1024  1694000000
```

← metric name + labels + value + timestamp

7. WHERE PROMETHEUS FITS IN DEVOPS AND SRE

- Used by DevOps engineers to monitor infrastructure, applications and CI/CD.
- Used by SREs to ensure reliability, availability and performance.
- Commonly integrated with Grafana for visualization.
- Works well in cloud, on-premises and Kubernetes environments.

8. SIMPLE FLOW

Here is a simplified view of how Prometheus fits in:

JUNIOR ENGINEER TIP

Monitoring is not just about tools. It is about understanding your system, knowing what good looks like, and being alerted when it deviates from normal.

VERIQTA

VERIQTA

How Prometheus Works

ARCHITECTURE, COMPONENTS AND DATA FLOW

METRICS TODAY RELIABLE SYSTEMS TOMORROW

1. PROMETHEUS ARCHITECTURE

Prometheus uses a pull-based model to collect time-series metrics from targets, stores the data locally, allows powerful querying with PromQL, and integrates with Alertmanager and Grafana.

Simple. Reliable. Scalable.

2. PROMETHEUS SERVER

- Central component
- Scrapes metrics from targets
- Stores data in a time series database (TSDB)
- Runs PromQL for querying
- Evaluates alerting rules
- Exposes a web UI (default port 9090)

Prometheus server is the engine of the monitoring system.

3. TARGETS

- Anything that exposes metrics via an HTTP /metrics endpoint
- Examples: servers, applications, containers, Kubernetes nodes, databases

Targets are the systems you want to monitor.

4. EXPORTERS

- Collect metrics from systems that do not export them natively
- Convert system metrics to a format Prometheus can scrape
- Examples: Node Exporter (Linux), cAdvisor (containers), MySQL Exporter, etc.

Exporters make more systems observable.

5. SERVICE DISCOVERY

- Automatically finds targets in dynamic environments
- Supports Kubernetes, cloud providers (AWS, Azure, GCP), DNS, and more
- Reduces manual configuration
- Keeps monitoring up to date as infrastructure changes

Service discovery helps Prometheus find and monitor targets automatically.

6. TIME SERIES DATABASE (TSDB)

- Stores metrics as time series data (metric name + labels + timestamp + value)
- Optimized for fast write and query performance
- Handles large amounts of metrics efficiently
- Retention is configurable (e.g. 15 days by default)

TSDB stores your monitoring data locally on disk.

7. PROMQL

- Prometheus Query Language
- Used to query and analyze metrics
- Supports filtering, aggregation, rate calculations and more
- Used in dashboards, alerts and troubleshooting

PromQL turns your metrics into useful information.

8. ALERTMANAGER

- Handles alerts sent from Prometheus
- Groups, deduplicates and routes alerts
- Sends notifications to email, Slack, Telegram, webhooks and more
- Supports silences and inhibition rules

Alertmanager ensures the right people are notified about real problems.

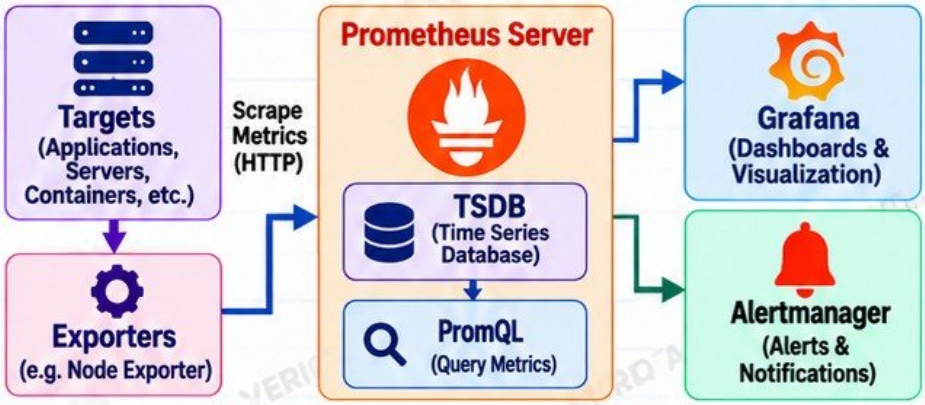
9. GRAFANA'S RELATIONSHIP WITH PROMETHEUS

- Grafana uses Prometheus as a data source
- Visualizes metrics with dashboards
- Helps you understand performance, trends and problems
- Does not store metrics (it queries Prometheus)

Grafana makes your metrics easy to see and understand.

10. OVERALL ARCHITECTURE

Here is a simplified view of how Prometheus works with its main components.



JUNIOR ENGINEER TIP

Prometheus does not replace logs or traces. It gives you metrics, which show you what is happening now. Use metrics, logs and traces together for full observability.

Learn Build Grow

@veriqta veriqta veriqta veriqta veriqta veriqta

FOLLOW VERIQTA FOR MORE DEVOPS NOTES

Installing and Running Prometheus

GET A WORKING PROMETHEUS INSTANCE IN MINUTES

SIMPLE SETUP
REAL SKILLS
PRODUCTION READY

1. INSTALLATION OPTIONS

You can run Prometheus in different ways depending on your environment and goals.

1. Binary (Linux/macOS/Windows)
2. Docker (simplest for beginners)
3. Package managers
4. Kubernetes (later in the course)
5. Cloud or managed services (e.g. Amazon Managed Service for Prometheus)

For this page, we will focus on local and Docker setup.

2. RUNNING PROMETHEUS LOCALLY

1. Download the latest version from the official website.

```
https://prometheus.io/download/
```

2. Extract the file.

```
tar -xvf prometheus-*.tar.gz
cd prometheus-*/
```

3. Start the server.

```
./prometheus
```

4. Access it in your browser.

```
http://localhost:9090
```

3. DOCKER-BASED SETUP

Docker is the easiest way to get started.

1. Pull and run Prometheus.

```
docker run -d \
  --name prometheus \
  -p 9090:9090 \
  -v $(pwd)/prometheus.yml:
 /etc/prometheus/prometheus.yml \
  prom/prometheus
```

2. Check if the container is running.

```
docker ps
```

3. View logs (if needed).

```
docker logs -f prometheus
```

4. DEFAULT PORT 9090

- Prometheus runs on port 9090 by default.
- You can change it with the --web.listen-address flag.
- Access the web interface at:

```
http://localhost:9090
```

5. PROMETHEUS WEB INTERFACE

Once running, open your browser and go to:

```
http://localhost:9090
```

You will see the Prometheus UI, which allows you to:

- Run PromQL queries
- View targets
- Check status and configuration
- Explore metrics

6. CHECKING SERVER HEALTH

1. Check if Prometheus is up.

```
curl http://localhost:9090/-/healthy
```

2. Check server status.

```
curl http://localhost:9090/-/ready
```

If you see "Prometheus is Healthy." your server is running correctly.

7. BASIC DIRECTORY STRUCTURE

After extracting the binary, you will see a structure like this:

```
prometheus/
├-- prometheus          # the server binary
├-- promtool            # CLI tool
├-- prometheus.yml      # configuration file
├-- consoles/           # UI consoles
└-- console_libraries/  # UI libraries
```

8. FIRST HANDS-ON PROMETHEUS INSTANCE

1. Install or run using Docker
2. Start the server
3. Open http://localhost:9990
4. Run a simple query (e.g. up)
5. View the Targets page
6. Explore the Status page
7. Confirm everything is working

YOU DID IT!

You now have a working Prometheus instance running. In the next pages, you will learn how to configure targets, collect real metrics, and write useful PromQL queries.

IMPORTANT

- This is a learning environment. Do not expose your Prometheus server to the internet.
- Use firewalls and authentication in production.

VERIQTA

VERIQTA

Understanding prometheus.yml

CONFIGURE SCRAPING, TARGETS AND HOW PROMETHEUS WORKS

SIMPLE CONFIG
RELIABLE MONITORING
REAL SKILLS

1. WHY PROMETHEUS NEEDS CONFIGURATION

Prometheus needs a configuration file to know:

- What to scrape (targets)
- How often to scrape
- Which jobs to monitor
- How to label and organize metrics
- How to evaluate alerting rules

The main configuration file is `prometheus.yml`.

2. GLOBAL CONFIGURATION

The global section controls settings that apply to the entire server.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
```

- `scrape_interval`: How often Prometheus scrapes targets.
- `evaluation_interval`: How often alerting rules are evaluated.

3. SCRAPE CONFIGS

The `scrape_configs` section defines what to scrape.

```
scrape_configs:
  - job_name: "node-exporter"
    static_configs:
      - targets:
          - "localhost:9100"
```

- `job_name`: A unique name for the job.
- `static_configs`: Manually defined targets.
- `targets`: The list of endpoints to scrape (host:port).

4. COMPLETE EXAMPLE prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
- job_name: "prometheus"
  static_configs:
  - targets: ["localhost:9090"]

- job_name: "node-exporter"
  static_configs:
  - targets: ["localhost:9100"]
```

Prometheus

You can add multiple jobs to monitor different systems.

5. YAML INDENTATION MISTAKES

YAML uses indentation. Incorrect indentation will cause errors.

Incorrect

```
scrape_configs:
- job_name: "node-exporter"
  static_configs:
    - targets:
        - "localhost:9100"
```

This will fail because the indentation is wrong.

Correct

```
scrape_configs:
 - job_name: "node-exporter"
   static_configs:
     - targets:
         - "localhost:9100"
```

6. VALIDATE CONFIGURATION

Use `promtool` to check if your configuration file is valid.

```
promtool check config \
  prometheus.yml
```

- This checks for syntax errors.
- Useful before starting or reloading Prometheus.
- Install `promtool` (comes with Prometheus).

7. RELOADING CONFIGURATION

You can reload the configuration without restarting the server.

```
curl -X POST \
http://localhost:9090/-/reload
```

- Changes take effect immediately.
- No downtime.
- Useful when you add new targets or update scrape intervals.

8. KEY SECTIONS SUMMARY

Section / Field	Purpose
global	Global settings for the server
scrape_interval	How often to scrape targets
evaluation_interval	How often to evaluate alert rules
scrape_configs	Defines targets to scrape
job_name	Unique name for a scrape job
static_configs	Manually defined targets
targets	List of endpoints to scrape

9. IMPORTANT NOTES

- Use correct YAML indentation.
- Validate with `promtool`.
- Reload instead of restarting.
- Keep your configuration organized and version controlled (e.g. Git).
- You can monitor multiple systems by adding more jobs.

JUNIOR ENGINEER TIP

Start with a simple configuration, get it working, then add more targets. Always validate your YAML before reloading to avoid silent mistakes.

VERIQTA

Targets and the Pull-Based Monitoring Model

HOW PROMETHEUS COLLECTS METRICS FROM YOUR SYSTEMS

OBSERVE
METRICS
DETECT ISSUES
KEEP SYSTEMS
RUNNING

1. WHAT IS A TARGET?

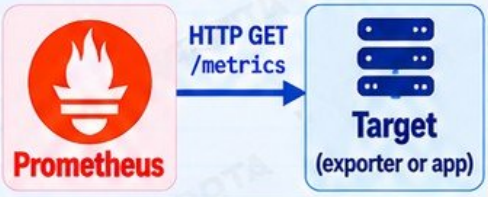
A target is any system that exposes metrics via an HTTP endpoint, usually `/metrics`. Examples:

- Servers (Node Exporter)
- Applications
- Databases
- Containers
- Kubernetes components
- Custom services

A target is simply something Prometheus scrapes.

2. WHAT DOES SCRAPING MEAN?

Scraping means Prometheus periodically sends an HTTP request to the target's `/metrics` endpoint and collects the latest metrics.



Prometheus pulls metrics from targets at a fixed interval.

3. PULL vs PUSH MONITORING

PULL (Prometheus)

- Prometheus pulls metrics
- Simpler architecture
- Targets expose `/metrics`
- Works well in dynamic environments
- More control and reliability

PUSH (Other systems)

- Targets push metrics to a server
- Used when targets cannot be scraped (e.g. short-lived jobs)
- More complex to manage
- Requires an additional component (e.g. Pushgateway)

4. SCRAPE INTERVALS

You define how often Prometheus scrapes each target.

```
scrape_configs:
  - job_name: "node-exporter"
    scrape_interval: 15s
    static_configs:
      - targets: ["localhost:9100"]
```

- Common intervals: 15s, 30s, 60s
- Shorter intervals = more data
- Longer intervals = less load

Choose an interval based on how important the metrics are and the load on your systems.

5. /METRICS ENDPOINTS

Targets expose metrics at the /metrics endpoint.

```
http://<target-ip>:<port>/metrics
```

- Returns metrics in text format
- Each line is a metric with labels and a value
- Example: http://localhost:9100/metrics

Example output:

```
node_cpu_seconds_total{mode="idle"} 12345
node_memory_MemTotal_bytes 1638420224
http_requests_total{method="GET"} 1024
```

6. TARGET HEALTH

In the Prometheus UI, each target shows a health status:

UP

- Prometheus can reach the target
- Metrics are being collected
- Everything is working

DOWN

- Prometheus cannot reach the target
- Metrics are not being collected
- There is a problem (network, configuration, or the target is down)

7. SCRAPE FAILURES

A target can be DOWN because of:

- Target is not running
- Wrong IP address or port
- /metrics endpoint not found (404)
- Network or firewall blocking access
- DNS resolution issues
- Timeout (target too slow)

Check the error message in the Prometheus UI for details.

8. TROUBLESHOOTING AN UNREACHABLE TARGET

1. Check if the target is running
   ```
   curl http://<target-ip>:<port>/metrics
   ```
2. Verify network connectivity
   ```
   ping <target-ip>
   ```
3. Check if the port is open
   ```
   telnet <target-ip> <port>  # or nc -zv <ip> <port>
   ```
4. Review Prometheus target status and error message
5. Fix the issue and verify the target is UP

9. WHY NETWORK CONNECTIVITY MATTERS

Prometheus must be able to reach the target over the network.

- Correct IP address or hostname
- Correct port
- Firewall allows access
- Security groups allow traffic
- DNS resolves correctly
- The target is listening
- No network policy blocking traffic

If Prometheus cannot reach the target, it cannot collect metrics.

JUNIOR ENGINEER TIP

When a target is DOWN, always check network connectivity and the `/metrics` endpoint first. Most issues are related to networking, incorrect ports, or the service not running.

VERIQTA

VERIQTA

Understanding Prometheus Metrics

THE BUILDING BLOCKS OF OBSERVABILITY

METRICS TELL THE STORY OF YOUR SYSTEM

1. METRIC NAMES

- Metric names identify what is being measured.
- Use a descriptive name with a consistent naming convention (usually snake_case).
- Example:

```
http_requests_total
```

The name tells you what the metric represents.

2. METRIC VALUES

- Metric values are numeric data points (usually integers or floats).
- They represent the measured value at a specific point in time.
- Example:

```
http_requests_total 1024
```

The value shows how much or how many.

3. TIMESTAMPS

- Every metric sample has a timestamp.
- Timestamps show when the value was recorded (Unix time in seconds).
- Example:

```
http_requests_total 1024
1694083200
```

This value was recorded at timestamp 1694083200 (UTC time).

4. LABELS

- Labels add context to metrics.
- They are key-value pairs that describe the metric (e.g. instance, job, method).
- Example:

```
http_requests_total{
 method="GET",
instance="web1:9100"
} 1024
```

Labels help you filter, group and analyze metrics.

5. TIME SERIES

- A time series is a metric name + a unique set of labels.
- Each unique label combination creates a different time series.
- Example:

```
http_requests_total{
 method="GET"}
http_requests_total{
  method="POST"}
```

These are two different time series.

6. SAMPLES

- A sample is a single data point in a time series.
- It contains a value and a timestamp.
- Over time, multiple samples build the full time series.
- Example (3 samples):

```
http_requests_total 1000 1694083000
http_requests_total 1024 1694083200
http_requests_total 1050 1694083400
```

Each line is one sample in the time series.

7. LABEL SETS

- A label set is the complete set of labels for ta metric.
- The label set uniquely identifies a time series.
- Example label set:

```
{job="node-exporter",
 instance="web1:9100",
 method="GET"}
```

Changing any label value creates a new time series.

8. WHY LABELS MAKE METRICS POWERFUL

- Labels allow you to slice and filter metrics.
- You can compare instances, environments, regions, etc.
- They enable powerful queries in PromQL.
- Example use cases:
  - Find errors per service
  - Compare CPU usage per host
  - Filter metrics by environment (prod, staging, dev)

Labels turn simple numbers into useful insights.

9. CARDINALITY INTRODUCTION

- Cardinality is the number of unique time series.
- High cardinality means too many unique label combinations.
- This can increase memory usage, slow down queries and affect performance.
- Examples of high-cardinality labels (avoid these):
  - User IDs (user_id="12345")
  - Request IDs
  - Timestamps in labels
  - Unbounded values (e.g. session_id, uuid)

KEEP LABELS MEANINGFUL AND CONTROLLED
Use labels that have a limited, predictable number of values.

10. READING A REAL METRIC CORRECTLY

```
node_cpu_seconds_total{mode="idle",instance="web1:9100",
job="node-exporter"}  12345.678  1694083200  1894083200}
```

Metric name | Labels (key=value pairs) | Metric value (12345.678) | Timestamp (1694083200)

What this tells us:

- Metric name: `node_cpu_seconds_total` (CPU time in seconds)
- Labels: `mode=idle`, `instance=web1:9100`, `job=node-exporter`
- Value: 12345.678 (total idle CPU seconds)
- Timestamp: 1694083200 (when it was recorded)

This is one sample in a time series. Over time, more samples are collected.

JUNIOR ENGINEER TIP

Good metric design and label usage make your monitoring system more useful, performant and easier to troubleshoot.

Learn Build Grow

VERIQTA @veriqta veriqta veriqta veriqta veriqta veriqta

FOLLOW VERIQTA FOR MORE DEVOPS NOTES

VERIQTA

VERIQTA

The Four Prometheus Metric Types

UNDERSTAND WHEN TO USE EACH TYPE AND REAL EXAMPLES

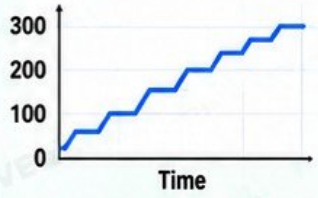
RIGHT METRICS BETTER OBSERVABILITY STRONGER SYSTEMS

1. COUNTER

- A counter is a metric that only goes up.
- It represents a cumulative value (total).
- It never decreases (except on restart).
- Used for counting events.

Common use cases:

- HTTP requests
- Errors
- Tasks completed
- Bytes transferred



Counter (only increases)

2. GAUGE

- A gauge can go up and down.
- It represents a value at a point in time.
- Used for measurements that change.

Common use cases:

- CPU usage
- Memory usage
- Active connections
- Queue length
- Temperature

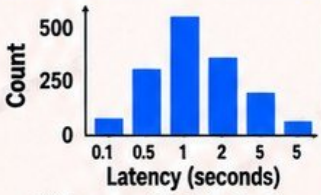
Gauge (goes up and down)

3. HISTOGRAM

- A histogram measures distributions of values (e.g. request duration).
- It buckets observations into configurable ranges (buckets).
- Provides count, sum and buckets.
- Useful for understanding the distribution of values.

Common use cases:

- Request latency
- Response size
- Processing time



Histogram (distribution)

4. SUMMARY

- A summary also measures distributions of values.
- Calculates quantiles (e.g. 50th, 90th, 99th percentiles).
- Good for client-side instrumentation.
- Does not expose buckets (like histogram).

Common use cases:

- Request latency (quantiles)
- Application-level metrics

Quantile	Value (seconds)
50%	0.120
90%	0.450
99%	1.230

Summary (quantiles)

5. WHEN EACH SHOULD BE USED

Metric Type	Use When
Counter	You need to count something that only increases (events).
Gauge	You need to track a value that goes up and down.
Histogram	You need to understand the distribution of values (latency, sizes, etc.).
Summary	You need quantiles and are instrumenting at the application level.

6. COUNTER EXAMPLE: HTTP REQUESTS

Counts total HTTP requests received.

```
http_requests_total{
  method="GET",
  code="200",
  instance="web1:9100"
}  12345
```

This means the server has received 12,345 HTTP GET requests with status code 200.

7. GAUGE EXAMPLE: MEMORY USAGE

Shows current memory usage.

```
node_memory_MemAvalable_bytes{
  instance="node1:9100",
  job="node-exporter"
} 7340032000
```

This means the node has 7,340,032,000 bytes (about 7.3 GB) of available memory at this moment.

8. HISTOGRAM EXAMPLE: REQUEST LATENCY

Tracks the distribution of request duration.

```
http_request_duration_seconds_bucket{le="0.1"} 240
http_request_duration_seconds_bucket{le="0.5"} 892
http_request_duration_seconds_bucket{le="1"} 1240
```

This shows how many requests completed within each latency bucket (≤ 0.1s, ≤ 0.5s, ≤ 1s).

9. COMMON BEGINNER MISTAKE

Treating counters like gauges

- Counters should not be used to track values that go up and down.
- Do not calculate average or current value directly from a counter.
- Use rate() or increase() for counters.
- If a metric can decrease, it is not a counter.

JUNIOR ENGINEER TIP

- Choose the right metric type based on what you want to measure.
- Use meaningful metric names and labels.
- Use histograms or summaries for latency.
- Use rate() for counters in PromQL queries.
- Monitor metric types in your applications and infrastructure.

VERIQTA @veriqta veriqta veriqta veriqta veriqta veriqta

FOLLIOW VERIQTA FOR MORE DEVOPS NOTES

Learn Build Grow

VERIQTA

LEARN | PRACTICE | BUILD | GROW

VERIQTA

Exporters and Node Exporter

COLLECT SYSTEM METRICS FROM LINUX SERVERS

MONITOR INFRASTRUCTURE
BUILD STABILITY
GAIN VISIBILITY

1. WHY EXPORTERS EXIST

- Prometheus uses a pull model. It scrapes `/metrics` endpoints from targets.
- Many systems do not expose metrics natively.
- Exporters collect system or application metrics and expose them in a format Prometheus can scrape.
- Exporters bridge the gap between your infrastructure, applications and Prometheus.

Without exporters, Prometheus would not be able to collect metrics from most systems.

2. WHAT NODE EXPORTER DOES

- Node Exporter collects hardware and OS-level metrics from Linux servers.
- It exposes metrics at the `/metrics` endpoint.
- It provides metrics about CPU, memory, filesystem, network, load, processes and more.
- It is lightweight, reliable and widely used in production.

Node Exporter helps you monitor the health and performance of your Linux servers.

3. INSTALLING NODE EXPORTER

1. Download the latest version

```
wget https://github.com/prometheus/
node_exporter/releases/latest/
download/node_exporter-*.linux-amd64.tar.gz
```

2. Extract and install

```
tar -xvf node_exporter-*.linux-amd64.tar.gz
sudo mv node_exporter-*/node_exporter
/usr/local/bin/
```

3. Create a user (recommended)

```
sudo useradd --no-create-home \
  --shell /bin/false node_exporter
```

4. Run as a service (systemd)

```
sudo tee /etc/systemd/system/node_exporter.service
[Unit]
Description=Prometheus Node Exporter
[Service]
ExecStart=/usr/local/bin/node_exporter
Restart=always
[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
```

4. DEFAULT PORT 9100

- Node Exporter runs on port 9100 by default.
- You can check it with:

```
ss -tulnp | grep 9100
```

- Access the metrics in your browser:

```
http://<server-ip>:9100/metrics
```

If you see metrics output, Node Exporter is running correctly.

5. KEY METRICS COLLECTED

CPU
- node_cpu_seconds_total
- node_load1
- node_cpu_info

Memory
- node_memory_MemTotal_bytes
- node_memory_MemAvailable_bytes
- node_memory_Active_bytes

Filesystem
- node_filesystem_size_bytes
- node_filesystem_free_bytes
- node_filesystem_avail_bytes

Network
- node_network_receive_bytes_total
- node_network_transmit_bytes_total
- node_network_errors_total

6. ADDING NODE EXPORTER AS A TARGET

1. Edit your prometheus.yml

```
scrape_configs:
  - job_name: "node-exporter"
    static_configs:
      - targets: ["192.168.1.10:9100"]
```

2. Reload Prometheus

```
curl -X POST http://localhost:9990/-/reload
```

3. Check the target in Prometheus UI

Go to `http://localhost:9090/targets`
You should see the target UP.

7. LAB: MONITOR A LINUX SERVER

1. Install Node Exporter on a Linux server.
2. Verify it is running on port 9100.
3. Add it as a target in Prometheus.
4. Check the target status is UP.
5. Explore key metrics in the Prometheus UI:

```
node_cpu_seconds_total
node_memory_MemAvailable_bytes
node_filesystem_avail_bytes
node_network_receive_bytes_total
```

6. (Optional) Visualize the metrics in Grafana.

8. EXAMPLE METRICS OUTPUT

```
# HELP node_cpu_seconds_total Seconds total of CPU time s
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
node_cpu_seconds_total{cpu="0",mode="user"} 5432.10

# HELP node_memory_MemAvailable_bytes Memory available.
# TYPE node_memory_MemAvailable_bytes gauge
node_memory_MemAvailable_bytes 7340032000

# HELP node_filesystem_avail_bytes Filesystem space avail
# TYPE node_filesystem_avail_bytes gauge
node_filesystem_avail_bytes{mountpoint="/"} 50212352000
```

Each line is a metric with labels and a value.

9. IMPORTANT NOTES

- Node Exporter is for Linux servers.
- It has very low resource usage.
- Keep it updated.
- Secure it (firewall or security groups).
- Do not expose it publicly to the internet.
- Use proper labels for your targets (e.g. environment, role, region).
- Combine with Grafana for visualization and alerting.

JUNIOR ENGINEER TIP

Start with one server, get it working, then roll it out to all your servers.

VERIQTA

LEARN | PRACTICE | BUILD | GROW

VERIQTA

PromQL Fundamentals

QUERY YOUR METRICS. FIND ANSWERS. SOLVE PROBLEMS.

SIMPLE QUERIES
REAL INSIGHTS
BETTER SYSTEMS

Prometheus

1. WHAT PROMQL IS

- PromQL (Prometheus Query Language) is the language used to query and analyze time series data in Prometheus.
- It allows you to select, filter, aggregate and perform calculations on your metrics.
- Used in the Prometheus UI, dashboards (Grafana) and alerting rules.

PromQL turns your raw metrics into useful information.

2. INSTANT VECTORS

- An instant vector returns the latest value for each time series at a single point in time.
- Most basic query type.
- Used for current state / real-time values.

Example:

```
node_cpu_seconds_total
```

Returns the latest value for all matching time series.

3. RANGE VECTORS

- A range vector returns a range of data points over a period of time.
- Used with functions like `rate()`, `increase()`, `avg_over_time()`.
- Specified using [time], e.g. `[5m]`, `[1h]`.

Example:

```
node_cpu_seconds_total[5m]
```

Returns all samples from the last 5 minutes for each time series.

4. SCALARS

- A scalar is a single numeric value (e.g. 5, 100, 3.14).
- Used in calculations with vectors.

Example:

```
5
```

You can use scalars with operators:

```
node_cpu_seconds_total / 5
```

5. SELECTING A METRIC

- To select a metric, use its name directly.

Example:

```
up
```

- This returns the latest value of the up metric for all time series.
- You can also select multiple metrics using a regex:

```
{__name__=~"node_.*"}
```

6. FILTERING WITH LABELS

- Use label matchers to filter time series.
- Syntax: `metric{label="value"}`
- Common operators:

Operator	Meaning
=	Equal to (exact match)
!=	Not equal to
=~	Regex match (matches pattern)
!~	Regex not match

Examples:

```
up{job="node-exporter"}
up{instance!="localhost:9100"}
node_memory_MemTotal_bytes{instance=~"web.*"}
```

7. TIME RANGES

- Use [duration] to specify a time range for range vectors.
- Common durations:

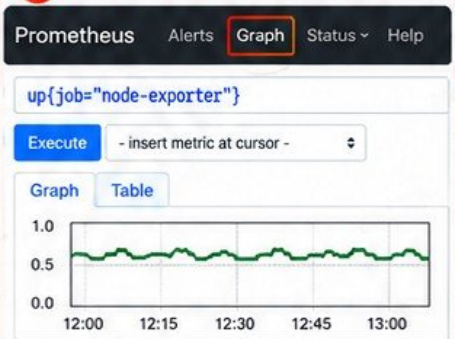
Duration	Meaning
1m	1 minute
5m	5 minutes
15m	15 minutes
1h	1 hour
6h	6 hours
1d	1 day
7d	7 days

Examples:

```
rate(http_requests_total[5m])
avg_over_time(node_cpu_seconds_total[1h])
```

8. RUNNING QUERIES IN THE PROMETHEUS UI

1. Open Prometheus in your browser `http://localhost:9090`
2. Go to the Graph tab.
3. Enter your query.
4. Click Execute.
5. View the results as Graph or Table.



9. BUILDING QUERIES INCREMENTALLY

1. Start with the metric name (e.g. node_cpu_seconds_total)
2. Add label filters (e.g. {instance="server1"})
3. Add a time range if needed (e.g. [5m])
4. Apply a function (e.g. rate(), sum(), avg_over_time())
5. Refine and adjust based on results

Example progression:

```
node_cpu_seconds_total
node_cpu_seconds_total{instance="server1"}
rate(node_cpu_seconds_total{instance="server1"}[5m])
sum(rate(node_cpu_seconds_total{instance="server1"}[5m]))
```

JUNIOR ENGINEER TIP

Start simple, test each step, and understand what each part of the query is doing.

VERIQTA

VERIQTA

PromQL for Real Monitoring

TURN METRICS INTO MEANINGFUL INSIGHTS

REAL QUERIES
REAL ANSWERS
BETTER
MONITORING

1. rate()

- Calculates the per-second average rate of increase for a counter over a time range.
- Used for most monitoring queries on counters.

```
rate(http_requests_total[5m])
```

Best for dashboards, alerting and general use.

2. irate()

- Calculates the per-second rate using only the last two data points.
- More sensitive to short-term changes.
- Useful for fast-moving metrics.

```
irate(http_requests_total[5m])
```

Use for detecting sudden spikes.

3. increase()

- Shows the total increase in a counter over a time range.
- Returns the number of events that occurred.

```
increase(http_requests_total[5m])
```

Useful for seeing total requests, errors, etc. over a period.

4. sum()

- Adds up all the values in a vector.
- Often used with **by** or **without** to group results.

```
sum(http_requests_total)
```

Use to get total values across instances or services.

5. avg()

- Calculates the average value.
- Frequently used with **by** or **without**.

```
avg(node_memory_MemAvailable_bytes)
```

Use to find average resource usage across multiple instances.

6. min()

- Returns the minimum value.
- Useful to find the lowest value across instances.

```
min(node_cpu_seconds_total)
```

Use to identify the lowest value (e.g. least used CPU).

7. max()

- Returns the maximum value.
- Useful to find the highest value across instances.

```
max(node_memory_MemAvailable_bytes)
```

Use to identify the highest value (e.g. most used memory).

8. count()

- Counts the number of elements in a vector.
- Often used with **by** or **without**.

```
count(up)
```

Use to count how many instances, targets or series match a condition.

9. by

- Groups the result by the specified label(s).
- Keeps only the listed labels in the output.

```
sum by (instance) (rate(http_requests_total[5m]))
```

Use to get results per instance, job, or other labels.

10. without

- Groups the result without the specified label(s).
- Removes the listed labels from the output.

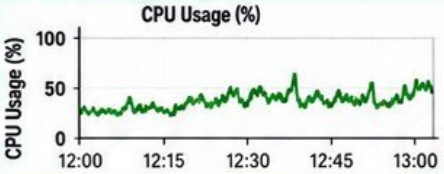
```
sum without (instance) (rate(http_requests_total[5m]))
```

Use when you want to aggregate across labels and remove specific dimensions.

11. CPU UTILIZATION EXAMPLE

- Calculate the CPU usage percentage for each instance.

```
100 - (avg by (instance)
 (rate(node_cpu_seconds_total{mode="idle"}[5m]))
 * 100)
```

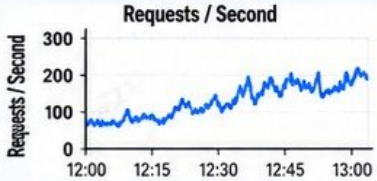


This shows the CPU utilization for each instance based on the non-idle CPU time.

12. HTTP REQUEST-RATE EXAMPLE

- Calculate the per-second rate of HTTP requests for each instance.

```
sum by (instance) (rate(http_requests_total[5m]))
```



This shows the number of HTTP requests per second for each instance using **rate()**.

13. WHY RAW COUNTERS OFTEN NEED rate()

- Counters only increase. They never decrease (except on restart).
- If you graph a raw counter, it will always go up, which is not useful for monitoring.
- Use **rate()** or **irate()** to calculate the per-second rate of increase.
- This gives you a meaningful view of activity over time (e.g. requests per second, errors per second).

Raw Counter (Not Useful)

```
http_requests_total
```

Always increasing, hard to interpret.

With rate() (Useful)

```
rate(http_requests_total[5m])
```

Shows the actual request rate over time.

VERIQTA

VERIQTA

Labels, Aggregation, and Cardinality

GET MORE VALUE FROM YOUR METRICS WITH THE RIGHT LABELS

GOOD LABELS
BETTER INSIGHTS
REAL IMPACT

1. WHY LABELS EXIST

- Labels add key-value pairs to metrics to identify dimensions like instance, job, environment or region.
- They allow you to filter, group and analyze metrics in a flexible way.
- The same metric name can have multiple time series, each with different label values.

Example metric with labels:

```
http_requests_total{
method="GET", job="web",
instance="web1", env="prod"}
```

2. FILTERING BY LABELS

Use label matchers to select specific time series.

```
http_requests_total
http_requests_total{job="web"}
http_requests_total{method="GET"}
http_requests_total{env="prod",
    job="web"}
```

Common label matchers:

=	Equal to
!=	Not equal to
=~	Regex match
!~	Regex not match

3. AGGREGATING ACROSS INSTANCES

Use aggregation functions to combine data from multiple instances.

```
sum(http_requests_total)
avg(http_requests_total)
min(http_requests_total)
max(http_requests_total)
count(http_requests_total)
```

Example: total requests for web job

```
sum(http_requests_total{job="web"})
```

4. GROUPING WITH BY

Use **by** to keep specific labels when aggregating.

```
sum(http_requests_total)
   by (job)
sum(rate(http_requests_total[5m]))
   by (instance, job)
```

Example:

```
sum(http_requests_total) by (job)
```

Shows total requests for each job.

5. REMOVING DIMENSIONS WITH WITHOUT

Use **without** to remove specific labels from the result.

```
sum(http_requests_total)
   without (instance)
```

Example:

```
sum(http_requests_total)
  without (instance, pod)
```

Removes instance and pod labels from the result.

6. HIGH CARDINALITY

- Cardinality is the number of unique time series created by label combinations.
- High cardinality means too many unique label values.
- This increases memory usage, storage, and query time.
- It can make Prometheus slow or unreliable.

Example of high cardinality:

```
http_requests_total{user_id="12345"}
```

Each unique user_id creates a new time series.

7. WHY USER IDs AND REQUEST IDs ARE DANGEROUS

- User IDs, session IDs, request IDs and other unique values create a new time series for every value.
- This can result in millions of time series.
- It consumes disk space, memory and CPU.
- It makes queries slow and expensive.
- It can cause Prometheus to run out of resources.

8. MEMORY AND PERFORMANCE CONSEQUENCES

- More time series = more memory usage.
- Higher disk storage requirements.
- Slower queries and dashboard loading.
- Increased CPU usage.
- Can lead to missed scrapes and instability.
- May require scaling the Prometheus server earlier than expected.

9. PRODUCTION LABEL DESIGN

- Use low cardinality, meaningful labels.
- Choose labels that help with monitoring and troubleshooting.
- Avoid user IDs, request IDs, session IDs, and random values.
- Use standardized label names (e.g. job, instance, env, region).
- Keep label values consistent.
- Review your label strategy regularly.

JUNIOR ENGINEER TIP

Use labels to answer real operational questions, not to store unique identifiers. Good labels give you insights. Bad labels create problems.

VERIQTA

VERIQTA

Monitoring Applications

INSTRUMENT YOUR APPLICATIONS. GET REAL INSIGHTS.

METRICS TURN APPLICATIONS INTO OBSERVABLE SYSTEMS

1. APPLICATION INSTRUMENTATION

- Application instrumentation means adding code to your application to expose metrics.
- It allows you to measure key behaviors such as requests, errors, and latency.
- Most programming languages have Prometheus client libraries.
- Instrumentation is usually lightweight and easy to add.

2. CLIENT LIBRARIES

- Prometheus provides client libraries for many languages.
- They help you create and expose metrics in the correct format.
- Popular client libraries:
  - Python: `prometheus_client`
  - Java: `simpleclient`
  - Go: `client_golang`
  - Node.js: `client`
  - .NET: `prometheus-net`
  - Ruby: `client_ruby`

3. EXPOSING /METRICS

- Applications expose metrics on an HTTP endpoint, usually `/metrics`.
- Prometheus scrapes this endpoint at a configured interval.
- The endpoint returns metrics in a simple text format.

Example endpoint:

`http://your-app:8080/metrics`

4. KEY APPLICATION METRICS

Metric	What It Shows
Request count	Total number of requests received
Error count	Number of failed requests (e.g. 4xx, 5xx)
Request duration	How long requests take
Active requests	Number of requests currently in progress
Business metrics	Metrics specific to your application (e.g. signups, payments, orders)

5. RED METHOD

The RED method is a simple way to monitor the health of an application.

- **Rate** – how many requests are coming in?
- **Errors** – how many requests are failing?
- **Duration** – how long are requests taking?

If you monitor these three, you can quickly understand the health of most applications.

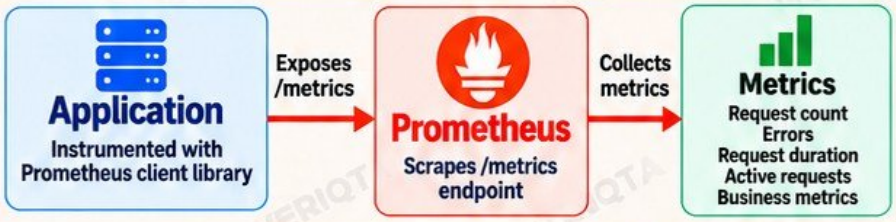
6. EXAMPLE METRICS (PYTHON)

Using the Python client library:

```
from prometheus_client import
Counter, Histogram, Gauge,
start_http_server

# Request count
REQUEST_COUNT = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'status']
)
# Request duration
REQUEST_DURATION = Histogram(
    'http_request_duration_seconds',
    'HTTP request duration',
    ['method']
)
# Active requests
ACTIVE_REQUESTS = Gauge(
    'http_requests_active',
    'Number of active requests'
```

7. APPLICATION → PROMETHEUS FLOW



8. EXAMPLE METRICS OUTPUT

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1025
http_requests_total{method="POST",status="500"}  12

# HELP http_request_duration_seconds HTTP request duration
# TYPE http_request_duration_seconds histogram
http_requetst_dursiten_seconds_bucket{le="0.1",method="GET"} 240
```

9. PRODUCTION TIPS

- Instrument key endpoints (not everything).
- Use meaningful metric names and labels.
- Avoid high-cardinality labels (e.g. user_id).
- Monitor business metrics that matter to your product.
- Set up dashboards and alerts for RED metrics.
- Test your /metrics endpoint after deployment.
- Keep metrics consistent across environments.

Better Engineers Build Better Systems

VERIQTA

JUNIOR ENGINEER TIP

Good application metrics help you detect problems early, understand user impact, and make better decisions in production.

VERIQTA

VERIQTA

Service Discovery and Dynamic Infrastructure

FIND AND MONITOR YOUR TARGETS AUTOMATICALLY

DYNAMIC
TARGETS
REAL
INFRASTRUCTURE
REAL SKILLS
VERIQTA

1. WHY STATIC TARGETS BECOME DIFFICULT

- In modern environments, servers and containers are created and destroyed frequently.
- IP addresses change.
- Manually updating prometheus.yml does not scale.
- You can easily miss new instances or keep monitoring old ones.
- Static targets work for small setups but become a maintenance problem in dynamic infrastructure.

2. WHAT IS SERVICE DISCOVERY?

- Service discovery automatically finds targets for Prometheus.
- It integrates with platforms like Kubernetes, cloud providers and container platforms.
- Prometheus discovers new targets, keeps them updated and removes old ones.
- This allows you to monitor dynamic environments without manual configuration.

3. DYNAMIC TARGETS

- Dynamic targets are services, pods, or instances that are automatically discovered.
- They can appear and disappear at any time.
- Examples: Kubernetes pods, EC2 instances, container services, microservices.
- Prometheus uses discovery mechanisms to keep the target list up to date.

4. CLOUD AND CONTAINER ENVIRONMENTS

AWS	Azure	GCP	Containers
• EC2 instances	• VMs	• Compute Engine	• Docker containers
• ECS services	• VMSS	• GKE	• Container labels
• Cloud tags	• Resource groups	• Labels	• Dynamic IPs
	• Tags	• Service accounts	

These environments are dynamic, so service discovery is essential for reliable monitoring.

5. KUBERNETES DISCOVERY INTRODUCTION

- In Kubernetes, pods and services change frequently.
- Prometheus can automatically discover:
  - Nodes
  - Pods
  - Services
  - Endpoints
- Kubernetes discovery uses the Kubernetes API.
- The Prometheus Operator makes this easier using ServiceMonitor and PodMonitor resources.

6. TARGET LABELS

- Discovered targets come with labels that provide metadata.
- Common labels include:

  job, instance, namespace, pod, service, node, region, environment

- These labels help you filter, group and organize your metrics.
- You can also add custom labels during relabeling.

7. RELABELING INTRODUCTION

- Relabeling allows you to modify target labels before scraping.
- You can:
  - Keep or drop targets
  - Rename labels
  - Add new labels
  - Extract values from labels
  - Filter based on label values
- Relabeling is powerful and helps you control exactly what gets scraped.

8. DISCOVER → SELECT → RELABEL → SCRAPE

Discover — Find targets automatically → **Select** — Choose which targets to keep → **Relabel** — Modify or add labels → **Scrape** — Collect metrics from targets

This flow happens continuously, so Prometheus always has an up to date list of targets.

9. WHY PRODUCTION ENVIRONMENTS NEED DISCOVERY

- Infrastructure is dynamic and constantly changing.
- Manual configuration does not scale.
- It reduces human error and ensures full coverage.
- New services are automatically monitored.
- Old targets are removed automatically.
- It is essential for Kubernetes, cloud, and large-scale environments.

JUNIOR ENGINEER TIP

Use service discovery to automatically monitor your real infrastructure. It saves time, reduces mistakes, and scales with your environment.

VERIQTA

VERIQTA

VERIQTA

Recording Rules

PRECOMPUTE QUERIES. FASTER DASHBOARDS. MORE RELIABLE MONITORING.

BETTER METRICS
BETTER INSIGHTS
A MORE RELIABLE YOU

1. WHAT ARE RECORDING RULES?

- Recording rules precompute the result of a PromQL query and store it as a new time series.
- They run at a defined interval and save the result with a metric name you choose.
- Instead of running the same complex query repeatedly, you query the precomputed metric.

2. WHY PRECOMPUTE COMPLEX QUERIES?

- Complex queries can be slow, especially over long time ranges.
- Precomputing reduces query load on the Prometheus server.
- Dashboards load faster.
- Alerts use precomputed metrics which are more reliable.
- It improves performance and provides consistent results for frequently used queries.

3. RULE GROUPS

- Rules are organized into groups.
- Each group has a name and an evaluation interval.
- You can include multiple recording rules in one group.

Basic structure:

```
groups:
  - name: example-rules
    interval: 30s
    rules:
      - record: <metric_name>
        expr: <promql_query>
```

4. NAMING RECORDED METRICS

- Use clear, consistent and descriptive names.
- Follow a naming convention (for example, use a prefix).
- Recorded metric names are just like normal metric names.
- Include details about what the metric represents.

Good example names:

- `app:http_requests:rate5m`
- `node:cpu_usage:avg5m`
- `kube:pod_memory_usage:sum`

5. EXAMPLE RECORDING RULE

```
groups:
- name: app-rules
  interval: 30s
  rules:
  - record: app:http_requests:rate5m
    expr: sum(rate(http_requests_total
     [5m])) by (job, instance)
```

Name of the recorded metric

PromQL query to calculate the value

This creates a new metric:
`app:http_requests:rate5m`
You can now use this metric in dashboards and alerts instead of the original query.

6. REUSING CALCULATED RESULTS

- Once recorded, you can use the new metric like any other metric.
- It can be used in Grafana dashboards, alerting rules, and other queries.
- This saves time and reduces load on the Prometheus server.

Example usage in a query:

```
app:http_requests:rate5m
{job="web"}
```

7. QUERY PERFORMANCE

- Recording rules reduce the need to run expensive queries repeatedly.
- They improve dashboard load times.
- They make alerts more stable.
- They help when you have high cardinality or long time ranges.
- Precomputing shifts the cost from query time to rule evaluation time.

Faster queries
Happier engineers
More reliable monitoring

8. PROMTOOL VALIDATION

- Use promtool to check your rule files for syntax errors before loading them.
- This helps you catch mistakes early.

```
promtool check rules rules_yml
```

Example output:

```
Checking rules.yml
 SUCCESS: 1 rules found
```

Always validate your rules before deploying them.

9. WHEN RECORDING RULES HELP

- ✓ Frequently used complex queries.
- ✓ Dashboards with slow queries.
- ✓ Alerting rules.
- ✓ High cardinality metrics.
- ✓ Large environments with many targets.
- ✓ Queries with long time ranges.

10. WHEN THEY CREATE COMPLEXITY

- ✗ Too many unnecessary rules.
- ✗ Recording simple queries that are already fast.
- ✗ Poorly named metrics.
- ✗ Rules that are never used.
- ✗ Creating rules for short-term or one-time analysis.
- ✗ Uncontrolled rule growth can be hard to maintain.

Use recording rules where they add real value, not everywhere.

JUNIOR ENGINEER TIP

Start with a few high-value recording rules. Focus on queries used in dashboards and alerts, then expand as needed.

VERIQTA

VERIQTA

Alerting Rules and Alertmanager

TURN METRICS INTO ACTION. GET NOTIFIED WHEN IT MATTERS.

RIGHT ALERTS
LESS NOISE
FASTER RESPONSE
MORE RELIABLE
SYSTEMS

1. METRICS vs ALERTS

METRICS

- Numerical data
- Collected over time
- Shows what is happening
- Used for dashboards, analysis and alerting
- **Example:**

```
cpu_usage 0.65
```

ALERTS

- Triggered when a condition is met
- Tells you when something needs attention
- Reduces manual monitoring
- Used for incident response
- **Example:**

CPU usage is above 80%

2. ALERTING RULES

- Alerting rules define conditions that evaluate metrics.
- When the condition is true, Prometheus creates an alert.
- You write rules in a YAML file (usually **alerts.yml**).

Example alerting rule:

```
- alert: HighCPUUsage
  expr: avg(rate(cpu_usage[5m])) > 0.8
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High CPU usage"
    description: "CPU usage is above
    80% for 5 minutes."
```

3. ALERT STATES

PENDING

- Condition is true, but for the specified time has not passed yet.
- Alert is not sent yet.

FIRING

- Condition has been true for the defined time (for).
- Alert is active and sent to Alertmanager.

RESOLVED

- Condition is no longer true. Alert stops firing.
- Alertmanager sends a resolution notification (if configured).

4. THE for DURATION

- The for clause specifies how long the condition must be true before the alert fires.
- It prevents false alerts from short spikes.

Example:

```
for: 5m
```

The condition must be true for 5 minutes before the alert fires.

5. LABELS AND ANNOTATIONS

- Labels add metadata to the alert (e.g. severity, team, environment).
- Annotations provide human-readable information (e.g. summary, description).

```
labels:
  severity: critical
  team: platform
  env: production
annotations:
  summary: "Instance is down"
  description: "{{ $labels.instance }}
  has been down for more than 5 minutes."
```

6. ALERTMANAGER OVERVIEW

- Alertmanager handles alerts sent from Prometheus.
- It manages:
  - **Routing** (where alerts go)
  - **Grouping** (combine similar alerts)
  - **Inhibition** (suppress related alerts)
  - **Silences** (temporarily mute alerts)
  - **Notifications** (email, Slack, PagerDuty, webhooks, etc.)
- It helps reduce noise and ensures the right people get the right alerts.

7. ROUTING, GROUPING, INHIBITION, SILENCES

Routing	Send alerts to the right teams based on labels (e.g. severity, service).
Grouping	Combine similar alerts to reduce noise.
Inhibition	Suppress certain alerts when a related alert is already firing.
Silences	Temporarily mute alerts (e.g. during maintenance).

8. WHY NOT EVERY THRESHOLD?

- Too many alerts create alert fatigue.
- Not every metric issue requires human action.
- Alerts should be actionable, meaning someone needs to do something when it fires.
- Focus on user impact, system health, and real problems.
- Use weekdays/weekend, time windows, and business context to avoid noise.

9. BEST PRACTICES

- Alert on symptoms, not raw metrics.
- Use meaningful labels.
- Set appropriate for durations.
- Avoid alerting on short spikes.
- Keep alert rules simple and clear.
- Document alerts and add runbooks.
- Review and tune alerts regularly.

JUNIOR ENGINEER TIP

Good alerts tell you what is wrong, why it matters, and what to do next.
Less noise means faster response and more reliable systems.

Monitor → Alert → Improve

VERIQTA

VERIQTA

VERIQTA

Building Actionable Production Alerts

RIGHT ALERTS. RIGHT PEOPLE. RIGHT ACTION. REAL IMPACT.

ALERT SMARTER
REDUCE NOISE
IMPROVE RELIABILITY

1. SYMPTOM-BASED ALERTS

Alert on symptoms, not just raw metrics.

- Focus on user impact and system behavior.
- Examples: high error rate, service down, high latency.

Good: Alert when the application is down.
Not good: Alert when CPU > 70%.

2. WARNING vs CRITICAL

- **Warning:** something is abnormal and needs attention soon.
- **Critical:** something is broken or causing user impact.
- Use different thresholds and notification channels.

Example (CPU):
- **Warning:** > 70% for 5 minutes
- **Critical:** > 90% for 5 minutes

3. AVOIDING ALERT FATIGUE

- Too many alerts make engineers ignore important ones.
- Avoid alerting on everything.
- Use meaningful thresholds and durations.
- Suppress non-actionable alerts.
- Group similar alerts.
- Regularly review and tune alerts.

Better alerts = faster response and a more reliable system.

4. CPU ALERT EXAMPLE

```
- alert: HighCPUUsage
  expr: avg(rate(cpu_usage[5m])) > 0.9
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "High CPU usage"
    description: "CPU usage is above
    90% for more than 5 minutes."
```

5. DISK-SPACE ALERT

```
- alert: LowDiskSpace
  expr: node_filesystem_free_
    bytes{mountpoint="/"} /
    node_filesystem_size_bytes{
    mountpoint="/"} < 0.1
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Low disk space"
    description: "Disk space
    is below 10% on /."
```

6. TARGET-DOWN ALERT

```
- alert: TargetDown
  expr: up == 0
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Instance down"
    description: "{{ $labels.
    instance }} has been
    down for more than
    2 minutes."
```

7. HIGH ERROR-RATE ALERT

```
- alert: HighErrorRate
  expr: rate(http_requests_total
    {status=~"5.."}[5m]) > 0.05
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High error rate"
    description: "More than 5%
    of requests are failing
    (5xx) for 5 minutes."
```

8. ALERT DURATION (for)

- The for clause specifies how long the condition must be true before the alert fires.
- It prevents alerts from temporary spikes.
- Use realistic durations (e.g. 2m, 5m).

Example:
`for: 5m`
Condition must be true for 5 minutes before the alert becomes active.

9. RUNBOOK LINKS

- Add runbook links in annotations to help with troubleshooting.
- Runbooks should include:
  - What the alert means
  - Possible causes
  - Step-by-step troubleshooting
  - Escalation path

Example annotation:
`runbook: "https://runbooks.veriqta.com/high-cpu"`

10. ACTIONABLE ALERT MESSAGES

- Use clear and concise messages.
- Include key information:
  - What is happening
  - Which service/instance
  - Severity level
  - What to do next
  - Runbook link

Good example:
"High CPU usage on prod-web-01 (> 90% for 5 minutes). Check runbook: https://runbooks.veriqta.com/cpu"

11. SRE PRINCIPLE

"Alert on conditions requiring human action."

- Not every threshold needs an alert.
- Only alert when human intervention is required.
- Let automation handle the rest.
- Focus on user impact, system availability, and real problems.

JUNIOR ENGINEER TIP

Good alerts are not about detecting every problem. They are about detecting the right problems, at the right time, with the right information, so engineers can take action.

VERIQTA

VERIQTA

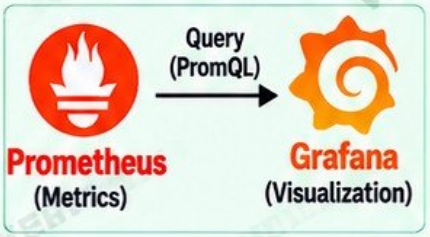
Prometheus and Grafana

COLLECT METRICS. VISUALIZE DATA. GET REAL INSIGHTS.

DATA VISIBILITY
BETTER DECISIONS
RELIABLE SYSTEMS

1. PROMETHEUS AS A DATA SOURCE

- Prometheus collects and stores time-series metrics.
- Grafana uses Prometheus as a data source to query and visualize these metrics.
- Grafana does not store metrics, it queries Prometheus in real time.



2. GRAFANA AS VISUALIZATION

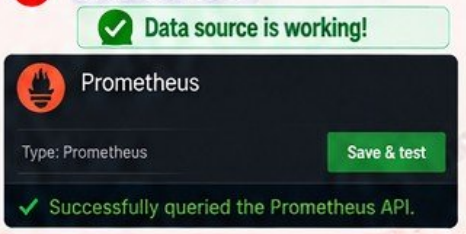
- Grafana turns metrics into beautiful, interactive dashboards.
- It helps you monitor systems, identify problems, and share insights.
- You can create dashboards for infrastructure, applications, and business metrics.

Key features:

- Visualize metrics
- Create dashboards
- Set up alerts (using Prometheus)
- Share dashboards
- Use variables and filters
- Supports multiple data sources

3. CONNECTING GRAFANA TO PROMETHEUS

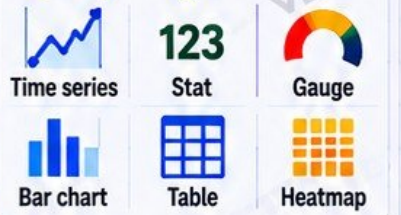
1. Open Grafana (`http://localhost:3000`)
2. Go to Connections → Data sources
3. Click Add data source
4. Select Prometheus
5. Enter the Prometheus URL (e.g. `http://localhost:9090`)
6. Click Save & test



4. PANELS

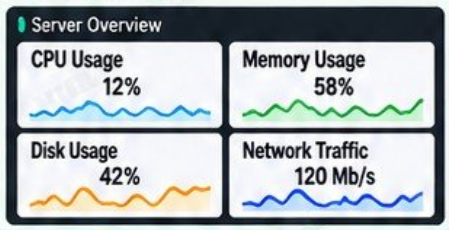
- Panels visualize your metrics as graphs, charts, tables, etc.
- Choose the right visualization type for your data.

Popular panel types:



5. DASHBOARDS

- Dashboards combine multiple panels into a single view.
- You can create dashboards for different teams and environments (e.g. Linux servers, Kubernetes, applications).



6. VARIABLES

- Variables allow you to filter and explore data dynamically.
- You can create dropdowns for items like:
  - instance
  - job
  - environment
  - cluster
  - region

7. USEFUL INFRASTRUCTURE DASHBOARDS

- Common dashboards to create:
- Linux server metrics (Node Exporter)
- Kubernetes cluster metrics
- Application metrics
- Database metrics
- Custom business metrics

8. KEY METRICS TO VISUALIZE

9. DASHBOARD VS ALERT

Dashboard

- Shows current and historical data
- Used for visualization and analysis
- Helps you understand the system
- Not a replacement for alerts
- Answer: "What is happening?"

Alert

- Notifies you when something is wrong
- Based on defined rules and thresholds
- Requires action when triggered
- Used with Alertmanager
- Answer: "What needs attention?"

10. AVOIDING BAD DASHBOARDS

- Do not create dashboards that show every possible metric.
- Avoid dashboards with too many panels, no clear purpose, or no context.
- Focus on the most important metrics.
- Use meaningful titles, labels, and thresholds.
- A good dashboard tells a story and helps you make decisions.

"A dashboard should inform, not confuse."

Less Noise More Insight

JUNIOR ENGINEER TIP

Start with simple dashboards, focus on key metrics, and improve over time.
A clear dashboard saves time during incidents.

VERIQTA

VERIQTA

VERIQTA

Prometheus with Kubrenetes

OBSERVE YOUR CLUSTER. MONITOR YOUR WORKLOADS. KEEP PRODUCTION HEALTHY.

KUBERNETES METRICS
BETTER VISIBILITY
HIGHER RELIABILITY
STRONGER SYSTEMS

1. WHY KUBERNETES MONITORING IS DIFFERENT

- Kubernetes is dynamic. Pods and services are constantly created, updated and deleted.
- Traditional static monitoring does not work well.
- You need automatic discovery, continuous monitoring and dynamic configuration.
- You must monitor the cluster, the infrastructure and your applications.

Kubernetes changes quickly, so your monitoring must also be dynamic.

2. DYNAMIC WORKLOADS

- Pods are ephemeral and can be created or destroyed at any time.
- Services get new endpoints frequently.
- Scaling happens automatically (HPA, cluster autoscaler).
- Nodes can be added or removed.
- Monitoring must automatically find and adapt to these changes.

Dynamic infrastructure requires dynamic monitoring.

3. KEY KUBERNETES COMPONENTS TO MONITOR

Pods
Your application workloads.

Services
Access to your applications.

Nodes
The worker machines.

Namespaces
Organize and isolate resources.

Cluster components
API server, scheduler, controller manager, etc.

4. KUBERNETES SERVICE DISCOVERY

- Prometheus automatically discovers Kubernetes resources using the Kubernetes API.
- Targets are found using labels and annotations.
- No need to manually add or update targets.

Kubernetes service discovery keeps your monitoring up to date automatically.

5. kube-state-metrics

- Exposes metrics about Kubernetes objects (pods, deployments, services, etc.).
- Does not collect application metrics.
- Provides cluster state and resource information.

Example metrics:

```
kube_pod_info
kube_deployment_status_replicas
kube_service_info
```

6. NODE METRICS

- Use node-exporter to collect metrics from each node.
- Provides OS and hardware metrics such as:
  - CPU usage
  - Memory usage
  - Disk usage
  - Network traffic
  - Filesystem metrics

Node metrics help you monitor the health of your Kubernetes infrastructure.

7. APPLICATION METRICS

- Your applications expose metrics (e.g. /metrics) using client libraries.
- Prometheus discovers and scrapes these metrics automatically.
- Monitor key application metrics such as:
  - Request count
  - Error rate
  - Request duration
  - Active connections
- Use ServiceMonitor or PodMonitor to configure scraping.

8. PROMETHEUS OPERATOR INTRODUCTION

- The Prometheus Operator makes it easier to run and manage Prometheus in Kubernetes.
- It handles deployment, configuration and scaling.
- You define monitoring resources using Custom Resources (CRDs).

Benefits:
- Simplifies setup and management
- Automatically discovers targets
- Supports high availability
- Integrates with Alertmanager

9. SERVICEMONITOR AND PODMONITOR

- **ServiceMonitor:** monitors services based on labels.
- **PodMonitor:** monitors pods directly.
- Both tell the Prometheus Operator which targets to scrape.

Example (ServiceMonitor):

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
  - port: metrics
    interval: 30s
```

10. KUBERNETES → PROMETHEUS → GRAFANA / ALERTMANAGER

Kubernetes (Pods, Nodes, Services, Applications) → **Prometheus** (Collects Metrics) → **Grafana** (Visualization) → **Alertmanager** (Alerting)

- Kubernetes provides the targets
- Prometheus collects the metrics
- Grafana visualizes the data
- Alertmanager sends alerts

Better visibility. Faster response.
More reliable Kubernetes environments.

JUNIOR ENGINEER TIP

Monitor your cluster, your infrastructure and your applications.
Use the Prometheus Operator to simplify configuration and keep your monitoring always up to date.

VERIQTA

VERIQTA

VERIQTA

Troubleshooting Prometheus

FIND PROBLEMS. FIX ISSUES. KEEP YOUR MONITORING RELIABLE.

OBSERVE
TROUBLESHOOT
RESOLVE
LEARN
IMPROVE

1. TARGET SHOWING DOWN

- Target appears as DOWN in Prometheus targets page.
- Check if the service is running.
- Verify the correct port and /metrics path.
- Check network connectivity and firewall rules.

Go to:
`http://<prometheus-ip>:9090/targets`

2. CONNECTION REFUSED

- The target port is not open or the service is not listening.
- Check if the application is running.
- Verify the correct port.
- Check firewall and security group rules.

Test with:
`curl -v http://<target-ip>:8080/metrics`

3. TIMEOUT

- Prometheus cannot reach the target in time.
- Possible causes: network latency, firewall, everloaded service, or wrong endpoint.
- Check network path and security rules.
- Verify the target is reachable from the Prometheus server.

Test with:
`curl -v --max-time 5 http://<target-ip>:8080/metrics`

4. WRONG /METRICS PATH

- The /metrics endpoint path is incorrect.
- Some applications use a different path (e.g. /actuator/prometheus).
- Check the application documentation.
- Update the scrape_config with the correct path.

Example:
`metrics_path: /actuator/prometheus`

5. DNS FAILURE

- Prometheus cannot resolve the target hostname.
- Check DNS configuration on the Prometheus server.
- Verify the service name (if using Kubernetes).

Test with:
`nslookup <hostname>`
`dig <hostname>`

6. NETWORK/FIREWALL PROBLEMS

- Firewall or security group is blocking the connection.
- Verify inbound and outbound rules.
- Check if the port is open (e.g. 9100, 8080, 9090).
- Use tools like ping, telnet or nc to test.

Test with:
`nc -zv <target-ip> <port>`

7. INVALID YAML

- Configuration file has syntax errors.
- Prometheus will fail to start or ignore the invalid config.
- Validate your configuration before loading it.

Test with:
`promtool check config prometheus.yml`

8. MISSING METRICS

- Target is up but expected metrics are not showing.
- Check if the application actually exposes the metrics.
- Verify the correct /metrics path and labels.
- Ensure the application is instrumented properly.

Test with:
`curl http://<target-ip>:8080/metrics`

9. INCORRECT LABELS

- Metrics have unexpected or missing labels.
- This can break queries, dashboards and alerts.
- Check the metric output and label names.
- Ensure consistent labeling across instances.

Example:
`http_requests_total{job="app", instance="10.0.0.1"}`

10. SLOW PROMQL

- Queries take too long to execute.
- Can be caused by high cardinality, large time ranges, or complex queries.
- Use smaller time ranges and more specific filters.
- Consider recording rules for heavy queries.

Example (use a time range):
`rate(http_requests_total[5m])`

11. HIGH CARDINALITY

- Too many unique label values (e.g. user_id, request_id).
- Causes high memory usage, slow queries and storage pressure.
- Avoid using dynamic values as labels.
- Use meaningful, low-cardinality labels.

Bad: `http_requests_total{user_id="12345"}`
Good: `http_requests_total{endpoint="/api"}`

12. DISK/STORAGE PRESSURE

- Prometheus uses significant disk space for time-series data.
- Can be caused by high cardinality or long retention.
- Check disk usage and retention settings.
- Consider reducing retention or using remote storage.

Check disk usage:
`df -h`
Prometheus storage path:
`/var/lib/prometheus`

13. STEP-BY-STEP TROUBLESHOOTING FLOW

1. Check Target → 2. Check Network → 3. Check Endpoint → 4. Check Configuration → 5. Check Logs → 6. Check Query → 7. Verify It Works

14. JUNIOR ENGINEER TIPS

- Start with the basics (is the target up?).
- Use simple tools (curl, ping, nslookup).
- Check Prometheus logs for errors.
- Validate your configuration with promtool.
- Keep your queries simple.
- Monitor disk usage and cardinality.
- Document common issues and solutions.

> Good monitoring is not just about collecting metrics, but making sure your monitoring actually works.

VERIQTA

Measure → Troubleshoot → Improve → Repeat

LEARN TODAY
BUILD TOMORROW

@veriqta veriqta veriqta veriqta veriqta veriqta

VERIQTA

VERIQTA

Production Prometheus

FROM LEARNING TO PRODUCTION. BUILD. SECURE. SCALE.

MONITOR
SECURE
SCALE
BE RELIABLE
PRODUCTION READY

1. RETENTION

- Retention defines how long metrics are stored.
- Default is 15 days.
- Adjust based on disk space, use case and requirements.
- Longer retention = more storage and higher costs.

Set a retention period that balances your operational and business needs.

2. TSDB STORAGE

- Prometheus uses a time-series database (TSDB) to store metrics locally on disk.
- Optimized for fast writes and queries.
- Stores data in blocks (usually 2 hours).
- Old data is automatically compacted and deleted based on retention.

Monitor disk usage to prevent storage issues.

3. RESOURCE PLANNING

- Plan CPU, memory and disk based on:
  - Number of targets
  - Scrape interval
  - Retention period
  - Cardinality
  - Query workload

As your environment grows, monitor and adjust resources before you hit limits.

4. CARDINALITY MANAGEMENT

- High cardinality increases memory usage, disk usage and query time.
- Avoid using user IDs, request IDs or other unique values as labels.
- Use low-cardinality, meaningful labels (e.g. service, environment, instance).

Keep your metrics useful, not just available.

5. SECURITY AND AUTHENTICATION

- Restrict access to Prometheus and its endpoints.
- Use authentication and authorization (basic auth, OAuth, reverse proxy).
- Limit network access using firewalls and security groups.
- Encrypt traffic with TLS.

Do not expose Prometheus to the public internet.

6. PROTECTING MONITORING ENDPOINTS

- Protect /metrics endpoints from unauthorized access.
- Use network policies (Kubernetes) or firewall rules.
- Allow access only from Prometheus servers.
- Avoid exposing sensitive metrics.

Monitoring data can reveal sensitive information about your infrastructure.

7. BACKUPS AND OPERATIONAL CONSIDERATIONS

- Prometheus can be rebuilt from configuration, but important data may be needed for analysis.
- Backup Prometheus configuration files.
- Use persistent storage.
- Monitor disk usage, memory, and overall health.
- Have a plan for disaster recovery.

Treat your monitoring system as a critical production system.

8. FEDERATION AND REMOTE WRITE INTRODUCTION

- Federation allows one Prometheus server to scrape metrics from another Prometheus server.
- Remote write allows sending metrics to long-term storage systems (e.g. Thanos, Cortex, Mimir, or external systems).
- Useful for large, multi-cluster or multi-region environments.

Use these when you need to centralize metrics or store data for a longer period.

9. SCALING LIMITATIONS

- A single Prometheus server has limits.
- As the number of targets, metrics and queries grow, CPU, memory and disk become bottlenecks.
- High cardinality and long retention increase resource usage.

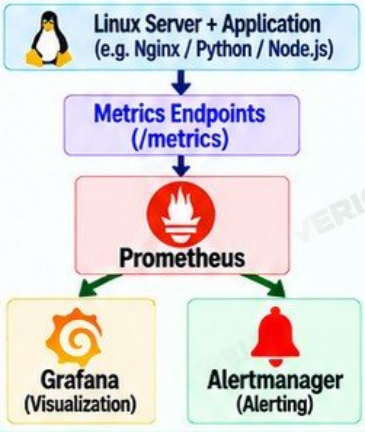
When you reach these limits, consider federation, remote write or a distributed solution.

10. WHEN ONE PROMETHEUS SERVER IS NO LONGER ENOUGH

- You may need multiple Prometheus servers when:
  - You have many clusters or regions
  - You have a high number of targets
  - You need long-term storage
  - You need higher availability
  - You have high query workloads

Plan for scale early to avoid performance and reliability issues later.

11. FINAL JUNIOR ENGINEER LAB



12. LAB TASKS

1. Configure Prometheus
2. Monitor Linux with Node Exporter
3. Scrape application metrics
4. Write PromQL queries
5. Create a recording rule
6. Create an actionable alert
7. Build a Grafana dashboard
8. Intentionally break one target
9. Diagnose the failure
10. Fix it
11. Verify monitoring has recovered

Complete this lab to apply everything you have learned and build real hands-on experience.

JUNIOR ENGINEER TIP

A well-designed, secure, and scalable Prometheus setup gives you visibility, faster troubleshooting, and a more reliable production environment.

Monitor
Learn
Improve
Grow

VERIQTA

@veriqta veriqta veriqta veriqta veriqta veriqta

LEARN TODAY
BUILD TOMORROW